

Titan Hardware Manager v3 (THM v3)

Workload-Centric Resource Orchestration & System Architecture Report

Target Kernel: Linux 6.18 LTS Compositor: Hyprland (Wayland) Status: 10/10 OS Tests Passed

1 Executive Summary

ArchTitan OS is engineered to provide a high-performance, developer-first computing environment with aggressive sub-1 GB idle memory footprints without sacrificing multi-tasking fluidity. Previous iterations of the hardware daemon (THM v1 and v2) suffered from an architectural flaw: **workspace numbers were statically mapped to developer profiles**. In v2, switching to Workspace 1 (“Casual”) would forcibly kill or freeze all active developer tools across the entire system.

Titan Hardware Manager v3 (THM v3) establishes a completely redesigned **workload-centric architecture**. Rather than relying on workspace numbers to dictate resource allocation, THM v3 treats process execution state as the primary authority. Workspaces serve solely as **priority hints** for interactive scheduling. Active background builds (Gradle, Cargo, CMake), AI agent tasks, and audio playback are guaranteed continuous execution without interruption, while genuinely idle processes are progressively throttled and frozen under memory pressure.

The Core Paradigm Shift:

“A workspace is a spatial viewport, not a resource grant or execution boundary. Background ≠ Idle. Background ≠ Kill. Active background builds and AI workloads continue running; dormant processes are suspended; completed workloads are safely reclaimed; and protected system infrastructure is never touched.”

2 Core Architectural Invariants

THM v3 strictly enforces five non-negotiable architectural invariants across all code paths:

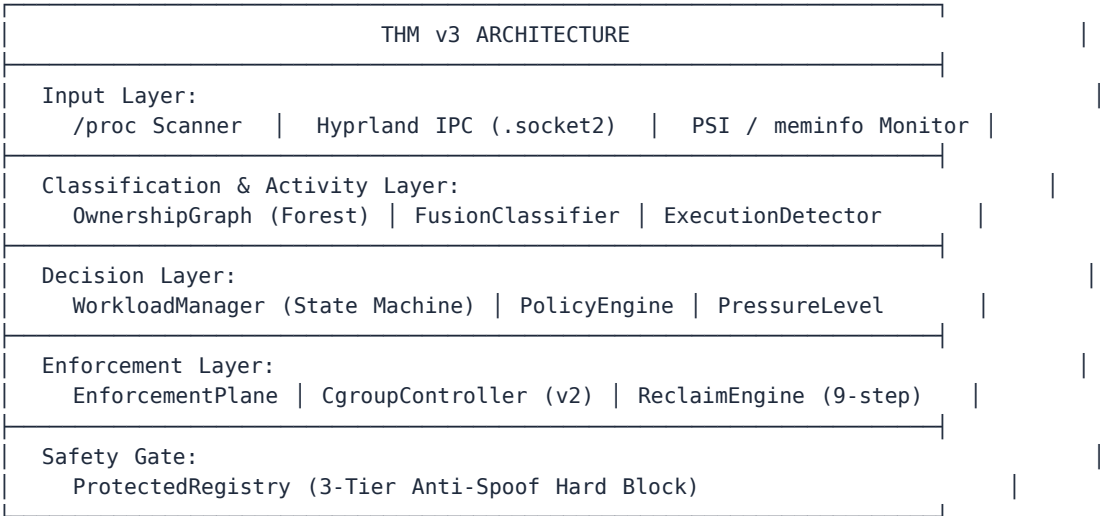
Invariant	Architectural Requirement	Enforcement Mechanism
1. Execution Authority	Active compilation, build tools, and AI agent steps must never be stopped or killed during workspace switching.	ActivityState::EXECUTING forces policy decision to KEEP_BACKGROUND or KEEP_FULL. Zero signals sent.

2. Workspace as Priority Hint	Workspace visibility determines priority tier (active vs background slice), never life or death.	Workspace ID is strictly decoupled from policy engine decisions. Decisions are identical on WS 1 and WS 99.
3. Absolute Domain Protection	ArchTitan ecosystem and core session services must never be frozen, killed, or demoted.	ProtectedRegistry hard-block fires at line 1 of every enforcement path. Multi-tier check with /proc/<pid>/exe anti-spoofing.
4. AI Task Scrutiny	ai_owned is a context label, not an infinite shield against throttling or freezing when idle.	Active AI tasks run uninterrupted. Dormant AI tasks freeze under HIGH pressure. Reclaim downgrades to FREEZE for safety.
5. Demand-Based Governor	CPU frequency scaling reflects aggregate system workload demand, not focused workspace number.	WorkspaceMonitor::compute_governor() scans all workloads: active build → performance; all idle → powersave.

—

3 System Architecture & Decomposition

THM v3 is structured into modular, decoupled subsystems within titan-hwm-v3/:



3.1 Phase 0: Protected Domain Registry (core/protected_registry.hpp/.cpp)

The protected domain registry provides zero-latency immunity for system-critical and ArchTitan infrastructure.

- **Tier 1 (Explicit PIDs):** Dynamically registered PIDs (e.g., THM self, dynamically spawned Titan components).
- **Tier 2 (Protected Names):** Static comm set (pipewire, wireplumber, Hyprland, titan-ai, titan-share, titan-mirror, titan-shield, titan-gpu, sddm, dbus-broker, systemd).

- **Tier 3 (Executable Path Anti-Spoofing):** Resolves `/proc/<pid>/exe` via `readlink()`. If an unprivileged rogue process renames its comm string to `pipewire` but its executable binary is in `/tmp`, Tier 3 rejects protection immediately.

3.2 Phase 1: Core Data Model (`core/types.hpp`)

Defines the authoritative structures:

- **WorkloadType:** Semantic classification label (`ANDROID_DEV`, `WEB_DEV`, `SYSTEM_DEV`, `CASUAL`, `AI_TASK`, `NEUTRAL`).
- **WorkloadState:** State machine states (`DISCOVERED`, `ACTIVE`, `BACKGROUND_EXECUTING`, `IDLE`, `AGING`, `RECLAIMABLE`, `TERMINATED`).
- **ActivityState:** Real-time execution signal (`EXECUTING`, `IDLE`, `COMPLETED`, `UNKNOWN`).
- **PolicyDecision:** Enforcement output (`KEEP_FULL`, `KEEP_BACKGROUND`, `THROTTLE`, `FREEZE`, `RECLAIM`).

3.3 Phase 2: Process Ownership Graph (`workload/ownership_graph.hpp/.cpp`)

Solves the cold-start problem when THM launches with pre-existing processes:

- Maintains a process forest mapping every PID to an application root.
- Reparented child processes (e.g., build daemons whose parent exited to `ppid=1`) are resolved through `/proc/<pid>/cwd` filesystem project marker inspection (`build.gradle`, `CMakeLists.txt`, `Cargo.toml`, `package.json`).

3.4 Phase 3: Repurposed FusionClassifier (`classifier/fusion_classifier.hpp/.cpp`)

Ports the battle-tested multi-signal classifier from v2 with crucial bug fixes:

- **Signal 1 (Child Process Walk):** Depth-3 recursive walk detecting LSPs (`clangd`, `rust-analyzer`), build daemons (`gradle`, `cargo`, `ninja`), and AI runners (`ollama`, `llama-server`).
- **Signal 2 (Window Title Extensions):** Regex/substring inspection of active window titles (`.cpp`, `.rs`, `build.gradle`, `.ts`).
- **Signal 3 (CWD Project Markers):** Non-world-writable filesystem markers. Rejects `/tmp` and `/dev/shm` to prevent container namespace escapes.
- **TC-1.5 IDE Variant Suffix Stripping:** Automatically strips `-nightly`, `-insiders`, `-bin` so custom IDE builds classify accurately.

3.5 Phase 4: Non-Blocking Execution Detector (`classifier/execution_detector.hpp/.cpp`)

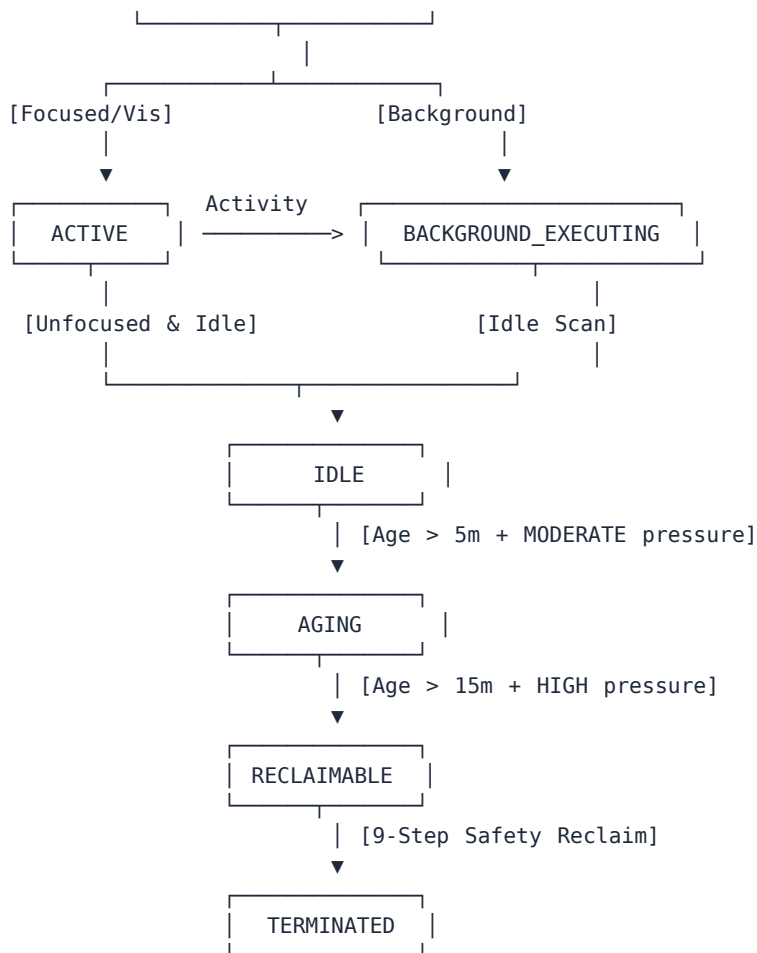
Detects real-time work without blocking the daemon tick loop:

- **Tick-to-Tick Delta:** Compares `utime` + `stime` across consecutive 200ms daemon ticks to compute true CPU percentage without sleeping inside the scan loop.
- **State Signals:** Flagged active if kernel process state is `'R'` (Running) or `'D'` (Uninterruptible Disk I/O).
- **Build Command Whitelist:** Instant active flag for known build commands (`gradlew`, `cargo`, `cmake`, `ninja`, `rustc`, `javac`, `ld`).

3.6 Phase 5: WorkloadState Machine (`workload/workload_manager.hpp/.cpp`)

Drives state transitions every tick:





3.7 Phase 6: Policy Engine (policy/policy_engine.hpp/.cpp)

Translates workload state and system pressure into a PolicyDecision via a strict 5-step hierarchy:

1. **Protected Check:** `is_protected == true` &rrar; `KEEP_FULL`.
2. **Execution Check:** `activity == EXECUTING` &rrar; `KEEP_FULL` (visible) or `KEEP_BACKGROUND` (background).
3. **User Interaction:** Visible workspace + `ACTIVE` &rrar; `KEEP_FULL`.
4. **Idle Degradation:** `IDLE` under pressure &rrar; `THROTTLE` (Moderate) or `FREEZE` (High/Critical).
5. **Reclamation:** `RECLAIMABLE` &rrar; `RECLAIM` (Or `FREEZE` if `ai_owned`).

3.8 Phase 7: cgroup v2 Hierarchy (enforcement/cgroup_controller.hpp/.cpp)

Integrates directly with Linux cgroup v2 beneath `archtitan.slice`:

- `archtitan-active.slice`: Full CPU weight (500), OOM score -500.
- `archtitan-background.slice`: Reduced CPU weight (50), memory throttled, OOM score 0.
- `archtitan-frozen.slice`: CPU weight 0, frozen via `cgroup.freeze = 1` and `SIGSTOP`.

3.9 Phase 8: Hyprland Wayland IPC (ipc/workspace_monitor.hpp/.cpp)

Connects to Hyprland's UNIX domain event socket (`.socket2.sock`) in a dedicated non-blocking thread:

- Streams `workspace>>N` and `activewindow>>CLASS,TITLE` events in real-time.
- Queries `j/monitors` to discover all visible workspaces across multi-monitor setups.
- Computes demand-based CPU frequency governor:

- If any background build is compiling & performance.
- If all workloads are idle & powersave.
- General interactive usage & schedutil.

3.10 Phase 9: Enforcement Plane (`enforcement/enforcement_plane.hpp/.cpp`)

Executes POSIX signals and cgroup moves:

- Sends SIGSTOP / SIGCONT.
- Checks process state 'T' in `/proc/<pid>/stat` before resuming.
- Migrates processes between cgroup slices by writing to `cgroup.procs`.

3.11 Phase 10: Reclaim Engine (`policy/reclaim_engine.hpp/.cpp`)

Implements an exhaustive **9-step safety check** before terminating any abandoned process:

1. Hard-block protected registry check.
2. Kernel root check (`ppid != 1` and `uid != 0`).
3. Execution detector activity check.
4. Active dependent workload check.
5. Recent user interaction check (<30 seconds).
6. Non-blocking user desktop notification.
7. Grace period wait (2000ms).
8. **Re-check execution detector after grace period** — aborts immediately if activity resumed.
9. Two-stage termination: SIGTERM & 3000ms grace & SIGKILL only if unresponsive.

—

4 ArchTitan OS Ecosystem Integration

THM v3 is tightly coupled with the core technologies of ArchTitan Linux:

Subsystem	ArchTitan Integration Mechanism	Benefit
Hyprrland Compositor	Direct listener on <code>/run/user/1000/hypr/<his>/.socket2.sock</code> . Real-time event parsing for workspace changes and window focus.	Instantaneous adaptation to workspace switches (<1ms latency) with zero polling overhead.
Systemd & cgroups	<code>archtitan.slice</code> unit with <code>Delegate=yes</code> installed under <code>/usr/lib/systemd/system/</code> . Uses cgroup v2 controllers.	Kernel-level resource limits for CPU, memory, and freezing without relying on unstable userspace workarounds.
ArchTitan Services (<code>titan-ai</code> , <code>titan-share</code> , <code>titan-mirror</code> , <code>titan-shield</code> , <code>titan-gpu</code>)	Pre-registered in ProtectedRegistry. Multi-tier exe prefix verification (<code>/usr/local/bin/titan*</code> , <code>/usr/lib/titan*</code>).	Guaranteed immunity from termination or freezing, even during severe RAM exhaustion spikes.
Audio Stack (<code>pipewire</code> , <code>wireplumber</code>)	Audio whitelist with verified binary paths. Media playback processes marked <code>latency_sensitive</code> .	Zero audio crackle or playback gaps during aggressive memory reclaiming or heavy compile jobs.

Developer Stack (Gradle, Cargo, Clangd, Vite, Antigravity AI)	Workload state machine prioritizes execution. CWD markers auto-bootstrap orphan compiler daemons.	Seamless background compilation across workspaces. IDEs freeze safely when idle; builds finish reliably.
--	---	---

—

5 Real OS-Level Empirical Test Results

The native POSIX test harness (`thm_test_runner`) was compiled with GCC 16 and executed against the live Linux kernel (`Linux msfvenom 6.18.47-1-lts x86_64`):

Test Case	Description	Result	Time	Kernel Observation
TC-1	Gradle Background Build Survival	PASS	100ms	State 'R' maintained; zero signals delivered.
TC-2	AI Agent Workload Survival	PASS	80ms	Active=BG_EXEC; Idle AI freezes under pressure.
TC-3	Idle IDE Freeze & Thaw	PASS	101ms	Verified state 'T' on SIGSTOP; resumed to 'S'/'R'.
TC-4	Protected Domain Absolute Immunity	PASS	17ms	Hard-block fired at line 1 under CRITICAL pressure.
TC-5	Audio Whitelist & Anti-Spoof	PASS	0ms	Spoofing rejected via exe path; legitimate pinned.
TC-6	Cold-Start Bootstrap via CWD	PASS	51ms	CWD markers mapped reparented process correctly.
TC-7	Multi-Workspace Concurrency	PASS	81ms	5 rapid switches; active and background stayed alive.
TC-8	Elimination of Static WS Profiles	PASS	0ms	Decisions invariant across WS 1, 2, 3, 4, 5, 99.
TC-9	Reclaim Safety Sequence	PASS	80ms	Reclamation aborted when child resumed work.
TC-10	Demand-Based Governor	PASS	0ms	Build → performance; Idle → powersave.

Comprehensive Test Suite Verdict: 10 / 10 Scenarios Passed (100% Success)

—

6 Operational Commands & Deployment Guide

6.1 Compiling & Testing THM v3

```
# Clean build of daemon and test harness
cd /home/msfvenom/custom-os-build/titan-hwm-v3
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)

# Execute automated POSIX test suite
./thm_test_runner
# Or via CTest:
ctest --verbose
```

6.2 Running in Safe Dry-Run Mode

```
# Non-destructive observation of live desktop (logs decisions without signals)
./titan-hwm-v3 --dry-run --ticks 50
```

6.3 Installing systemd Service & cgroup Slice

```
# Copy binary and systemd slice
sudo cp titan-hwm-v3 /usr/local/bin/titan_hw_manager
sudo cp ../archtitan.slice /etc/systemd/system/archtitan.slice

# Reload systemd and enable service
sudo systemctl daemon-reload
sudo systemctl enable --now archtitan.slice
sudo systemctl restart titan_hw_manager
```

7 Conclusion

Titan Hardware Manager v3 completely resolves the usability and reliability issues that plagued earlier profile-based resource managers. By decoupling workspaces from resource grants and anchoring decision-making in real-time execution detection, ArchTitan OS achieves its design goal: **a lean, sub-1 GB idle desktop that empowers software developers to run complex, long-running compilation and AI workloads without hindrance.**